

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника (по отраслям)

направленность (профиль)
«Технологии разработки программного обеспечения и обработки больших данных»

Курсовая работа
по дисциплине «Прикладные информационные технологии»

«Разработка и контейнеризация веб-приложения для отслеживания курсов валют в реальном времени с использованием Docker и автоматизация выпуска сертификатов Let's Encrypt»

Обучающегося 3 курса
очной формы обучения
Степановой Анны Андреевны

Руководитель курсовой работы:
старший преподаватель кафедры ИТЭО
Аксютин Павел Александрович

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
1.1. Контейнеризация и Docker	5
1.2. Микросервисная архитектура	5
1.3. WebSocket – протокол реального времени.....	7
1.4. Let's Encrypt и автоматизация SSL/TLS.....	8
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....	10
2.1. Архитектура приложения	10
2.2. Разработка веб-приложения	12
2.3. Контейнеризация и развёртывание	15
2.4. Результаты работы	16
2.5. Выводы по практической части	17
ЗАКЛЮЧЕНИЕ	19
СПИСОК ЛИТЕРАТУРЫ	21
ПРИЛОЖЕНИЕ А	23
ПРИЛОЖЕНИЕ Б.....	34

ВВЕДЕНИЕ

Современные веб-приложения всё чаще требуют функционирования в режиме реального времени, примером чего служат сервисы отображения курсов валют, биржевых котировок или системы мгновенного обмена сообщениями. Параллельно с этим возрастает потребность в надёжных и полностью автоматизированных подходах к развёртыванию подобных систем.

Платформа Docker стала де-факто стандартом для доставки и масштабирования приложений, однако настройка HTTPS-шифрования по-прежнему остаётся сложной задачей для многих разработчиков. Ключевая проблема заключается в том, что традиционный способ получения SSL-сертификатов требует ручного вмешательства, регулярного обновления вручную и жёсткой привязки сертификата к конкретному серверу.

Предлагаемым решением выступает использование связки из двух компонентов: `nginx-proxu` и `acme-companion`. Данная комбинация позволяет автоматически получать и своевременно обновлять сертификаты Let's Encrypt без необходимости остановки работающих сервисов и какого-либо участия администратора.

Актуальность данной работы обусловлена:

- Ростом числа микросервисных проектов
- Требованиями безопасности (HTTPS обязателен)
- Необходимостью автоматизации DevOps-процессов

Цель: разработать и контейнеризировать веб-приложение для отображения курсов валют в реальном времени с автоматическим получением HTTPS-сертификатов Let's Encrypt.

Задачи:

1. Разработать веб-приложение (Frontend + WebSocket + парсинг курсов)
2. Создать Docker-образы для всех компонентов
3. Написать файл docker-compose.yml для оркестрации
4. Настроить автоматический прокси и HTTPS через nginx-проху
5. Интегрировать acme-companion для выпуска сертификатов
6. Провести тестирование и задокументировать процесс

Объектом исследования в данной работе является процесс контейнеризации веб-приложений и автоматизация защиты передаваемых данных с помощью протоколов SSL/TLS.

Предметом исследования выступает совокупность программных инструментов, обеспечивающих контейнеризацию и автоматическое шифрование: платформа контейнеризации Docker, reverse-proxy NGINX-Проху, сервис автоматического получения сертификатов acme-companion, а также бесплатный центр сертификации Let's Encrypt.

Таким образом, в рамках работы рассматриваются не только теоретические аспекты контейнеризации, но и практическое применение указанных инструментов для развёртывания защищённого веб-приложения.

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1. Контейнеризация и Docker

Docker представляет собой технологическую платформу, предназначенную для упаковки программного обеспечения в изолированные контейнеры [3]. Среди ключевых достоинств такого подхода можно выделить унификацию среды выполнения на всех стадиях жизненного цикла приложения — от разработки до эксплуатации, простоту горизонтального масштабирования сервисов, а также воспроизводимость сборки, благодаря чему гарантируется идентичное поведение приложения вне зависимости от того, на каком оборудовании или в каком окружении оно запущено.

В экосистеме Docker можно выделить три базовых понятия: образ, контейнер и инструмент для оркестрации — Docker Compose [9]. Образ (Image) по своей сути является шаблоном, на основе которого создаётся контейнер. Он включает в себя всё необходимое для работы приложения: зависимости, библиотеки, настройки и сам код. Контейнер (Container) — это уже запущенный экземпляр образа, то есть работающее приложение, функционирующее в изолированном пространстве. Docker Compose служит для координации работы нескольких контейнеров: он позволяет описывать структуру многокомпонентного приложения в одном конфигурационном файле и управлять всеми его частями как единым целым [8].

1.2. Микросервисная архитектура

В рамках практической реализации применяется микросервисный подход, предполагающий разделение системы на независимые, слабо связанные компоненты, каждый из которых отвечает за строго определённую функцию [5]. Такая декомпозиция позволяет разрабатывать, масштабировать и обновлять отдельные части приложения независимо друг от друга, что особенно важно для систем, работающих в режиме реального времени. Разработанное веб-приложение для отслеживания курсов валют состоит из

трёх основных сервисов, каждый из которых запускается в собственном Docker-контейнере.

Сервис `currencysu-app` является центральным компонентом системы и выполняет функции backend-части приложения. Он отвечает за периодическое получение актуальных курсов валют от внешнего API Центрального банка Российской Федерации, а также за управление всеми WebSocket-соединениями с подключёнными клиентами. Внутри контейнера `currencysu-app` работает веб-сервер на базе Python и фреймворка Tornado, который одновременно обрабатывает HTTP-запросы для отдачи статических файлов (HTML, CSS, JavaScript) и поддерживает постоянные WebSocket-соединения для передачи данных.

Сервис `nginx-проху` [11] выполняет функцию обратного прокси-сервера (reverse-proxy), располагаясь между внешней сетью и внутренними сервисами приложения. Он принимает все входящие HTTP и HTTPS запросы от пользователей, после чего на основании имени домена, указанного в запросе, перенаправляет их на соответствующий контейнер. В данном случае `nginx-проху` настроен таким образом, что все запросы к домену `stepanna.bizml.ru` автоматически направляются на контейнер `currencysu-app` [2]. Помимо маршрутизации, этот сервис обеспечивает балансировку нагрузки в случае, если в будущем потребуется запустить несколько экземпляров `currencysu-app` для увеличения пропускной способности. Кроме того, именно `nginx-проху` выполняет терминацию SSL — расшифровку входящего HTTPS-трафика и последующее перенаправление уже незашифрованных запросов к внутренним сервисам, что освобождает прикладные компоненты от необходимости самостоятельно обрабатывать криптографические операции.

Сервис `асте-companion` предназначен для автоматического получения и своевременного обновления SSL-сертификатов от бесплатного удостоверяющего центра Let's Encrypt [10]. Этот компонент интегрируется с `nginx-проху` и отслеживает появление новых контейнеров, у которых в

переменных окружения указаны метки LETSENCRYPT_HOST и LETSENCRYPT_EMAIL. Когда обнаруживается контейнер currency-app с такими метками, asme-companion автоматически запускает процедуру оформления сертификата для соответствующего доменного имени [7]. Полученный сертификат помещается в специально отведённую директорию certs, откуда nginx-проху имеет возможность его загрузить и применять при обработке входящих HTTPS-запросов. Кроме того, asme-companion берёт на себя обязанность по регулярному продлению сертификата, осуществляя это каждые 60 дней в полностью автоматическом режиме, благодаря чему отпадает необходимость в ручном контроле сроков действия сертификатов и связанных с их истечением рисков.

Все три перечисленных сервиса объединены в одну внутреннюю сеть проху-network, что позволяет им взаимодействовать друг с другом, оставаясь при этом изолированными от внешней сети. Такая архитектурная организация не только повышает безопасность системы, но и упрощает её администрирование, поскольку все настройки маршрутизации, балансировки и шифрования сосредоточены в единственном компоненте — nginx-проху [4].

1.3. WebSocket – протокол реального времени

WebSocket обеспечивает полнодуплексный канал связи между клиентом и сервером. В отличие от HTTP, который требует новый запрос на каждое обновление, WebSocket позволяет серверу самому отправлять данные клиенту [12].

Для реализации механизма рассылки обновлений от сервера к клиентам в данной работе используется паттерн «Наблюдатель» (Observer). Этот поведенческий паттерн проектирования позволяет серверу, выступающему в роли наблюдаемого субъекта, автоматически уведомлять всех подключённых клиентов, которые являются наблюдателями, об изменении курсов валют без необходимости клиентам выполнять повторные запросы.

В контексте использования протокола WebSocket применение паттерна «Наблюдатель» даёт ряд существенных преимуществ. Прежде всего, удаётся полностью избавиться от нагрузки на сервер, связанной с механизмом опроса (polling). В классической схеме, где клиенты регулярно отправляют HTTP-запросы, им приходится непрерывно проверять, появились ли свежие данные, — даже в ситуациях, когда курсы валют остались прежними. Такой подход ведёт к неоправданному потреблению сетевого трафика, лишней нагрузке на процессор сервера и росту времени ответа.

Паттерн «Наблюдатель» даёт возможность серверу передавать данные исключительно в момент их фактического изменения. Это особенно ценно для приложений, работающих в режиме реального времени, так как обновления в них происходят не безостановочно, а через определённые промежутки времени. Также обеспечивается мгновенная рассылка обновлений всем подключённым клиентам.

Помимо этого, паттерн «Наблюдатель» обеспечивает простоту масштабирования системы. Добавление нового клиента сводится к созданию нового объекта-наблюдателя, связанного с его WebSocket-соединением, и регистрации этого наблюдателя у субъекта. При этом ни серверная логика получения данных от API ЦБ РФ [6], ни код, отвечающий за рассылку уведомлений, не требуют никаких изменений. Сервер продолжает работать в обычном режиме, не заботясь о том, сколько именно клиентов в данный момент подключено, что делает разработанную архитектуру гибкой и легко расширяемой.

1.4. Let's Encrypt и автоматизация SSL/TLS

Let's Encrypt представляет собой бесплатный центр сертификации, который полностью автоматизирует процесс выдачи SSL-сертификатов с помощью протокола ACME [1]. Работа данного сервиса организована следующим образом. Клиент, в роли которого в данном проекте выступает

асме-companion, направляет запрос на получение сертификата. После этого Let's Encrypt проверяет подтверждение владения доменом, используя один из доступных методов верификации, например HTTP-01 или DNS-01. При успешном прохождении проверки сертификат выдаётся сроком на 90 дней. При этом система автоматически инициирует его обновление через 60 дней после выдачи, что позволяет поддерживать HTTPS-защиту без какого-либо ручного вмешательства со стороны администратора.

Асме-companion в автоматическом режиме отслеживает все запущенные контейнеры, которые имеют метку LETSENCRYPT_HOST. При обнаружении таких контейнеров он самостоятельно инициирует процесс получения SSL-сертификата, а в дальнейшем также автоматически занимается его своевременным обновлением. Все полученные сертификаты сохраняются в общей директории ./certs, что позволяет другим сервисам, в частности nginx-проху, иметь к ним доступ.

Выбор связки nginx-проху и асме-companion обусловлен несколькими факторами. Прежде всего, это простота настройки: для полноценной работы достаточно указать всего четыре переменные окружения, что значительно снижает порог входа и минимизирует вероятность ошибок. Кроме того, nginx-проху и асме-companion зарекомендовали себя как надёжное и стабильное решение, активно используемое в коммерческой и учебной разработке. Немаловажным фактором является также широкая поддержка сообщества: оба инструмента имеют подробную документацию, регулярно обновляются, а при возникновении вопросов можно найти готовые решения типовых проблем на профильных форумах и в репозиториях GitHub.

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Архитектура приложения

Разработанное веб-приложение для отслеживания курсов валют Центрального банка Российской Федерации представляет собой клиент-серверную систему, функционирующую в контейнеризированной среде Docker.

Приложение состоит из следующих основных компонентов, представленных в таблице 1.

Таблица 1 — Основные компоненты веб-приложения

Компонент	Назначение
Клиентская часть (Frontend)	Веб-интерфейс для отображения курсов валют в реальном времени
Серверная часть (Backend)	Обработка запросов, получение данных от API ЦБ РФ, управление WebSocket-соединениями
Reverse-proxy (Nginx)	Маршрутизация трафика, терминация SSL, перенаправление HTTP на HTTPS
acme-companion	Автоматическое получение и обновление SSL-сертификатов от Let's Encrypt

Полная структура проекта представлена на рисунке 1.

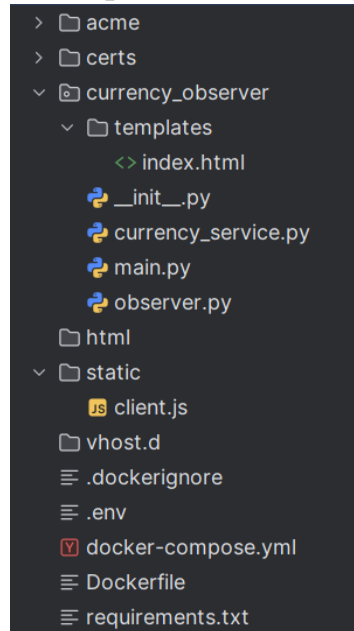


Рисунок 1. Архитектура веб-приложения

Взаимодействие компонентов системы осуществляется следующим образом:

1. Пользователь обращается к <https://stepanna.bizml.ru> через браузер.
2. Reverse-proxy (nginx-proxy) принимает запрос на порту 443 и, благодаря настроенному SSL-терминированию, расшифровывает HTTPS-трафик.
3. acme-companion автоматически обслуживает SSL-сертификаты, обеспечивая их своевременное обновление.
4. Reverse-proxy перенаправляет запрос на контейнер currency-app, который слушает внутренний порт 8888.
5. Серверная часть (Python/Flask):
 - Отдаёт клиенту HTML-страницу и JavaScript-файлы.
 - Устанавливает WebSocket-соединение по пути /ws для передачи данных в реальном времени.

- Периодически (каждые 30 секунд) обращается к внешнему API ЦБ РФ (http://www.cbr.ru/scripts/XML_daily.asp) для получения актуальных курсов валют.

6. Клиентская часть (JavaScript):

- Устанавливает WebSocket-соединение с сервером.
- Получает обновления курсов в реальном времени.
- Динамически обновляет таблицу на странице без необходимости её перезагрузки.

2.2. Разработка веб-приложения

Стек технологий, используемых в работе представлен ниже в таблице 2.

Таблица 2. Стек технологий разработки

Компонент	Технология	Назначение
Backend	Python 3.11 + Tornado	Веб-сервер, WebSocket, HTTP
API курсов	ЦБ РФ (XML)	Источник данных о курсах валют
Frontend	HTML5, CSS3, JavaScript	Пользовательский интерфейс
WebSocket	W3C WebSocket API	Двунаправленная связь клиент-сервер
Контейнеризация	Docker, Docker Compose	Упаковка и оркестрация
Reverse-proxy	Nginx (nginx-proxy)	Маршрутизация и SSL
SSL-сертификаты	Let's Encrypt, acme-companion	Бесплатные сертификаты

Серверная часть (Backend) состоит из файлов:

Файл `currency_service.py` — отвечает за получение курсов валют от API ЦБ РФ, на рисунке 2 показан краткий листинг кода.

```
1 import asyncio
2 import aiohttp
3 from typing import Dict, Any, Optional
4 from datetime import datetime, timedelta
5 import json
6
7
8 class CurrencyService: 3 usages
9     """Сервис для получения курсов валют с API ЦБ РФ"""
10
11     API_URL = "https://www.cbr-xml-daily.ru/daily_json.js"
12
13     def __init__(self, update_interval: int = 300) -> None:
14         self.update_interval = update_interval
15         self.last_update: Optional[datetime] = None
16         self.currencies: Dict[str, float] = {}
17
18     async def fetch_currency_rates(self) -> Dict[str, float]: 1 usage
19         """Получить курсы валют с API"""
20         try:
21             async with aiohttp.ClientSession() as session:
22
23                 headers = {
24                     'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
25                 }
26                 async with session.get(self.API_URL, headers=headers, timeout=10) as response:
27                     if response.status == 200:
```

Рисунок 2. Листинг файла `currency_service.py`

Файл `observer.py` — управляет WebSocket-соединениями и рассылкой обновлений, краткий листинг кода представлен на рисунке 3.

```
1 from typing import List, Dict, Any, Protocol, Optional
2 import json
3 import uuid
4 from datetime import datetime
5
6
7 class Observer(Protocol): 3 usages
8     """Протокол для наблюдателей"""
9     observer_id: str
10
11     async def update(self, currency_data: Dict[str, Any]) -> None: 1 usage
12         """Метод для обновления данных у наблюдателя"""
13         pass
14
15
16 class CurrencySubject: 4 usages
17     """Субъект, который отслеживает изменения курсов валют"""
18
19     def __init__(self) -> None:
20         self._observers: List[Observer] = []
21         self._currency_data: Dict[str, float] = {}
22
23     def attach(self, observer: Observer) -> None: 1 usage
24         """Добавить наблюдателя"""
25         if observer not in self._observers:
26             self._observers.append(observer)
27             print(f"Наблюдатель {observer.observer_id} подключен")
```

Рисунок 3. Листинг файла `observer.py`

Клиентская часть (Frontend) состоит из следующих файлов:

Файл client.js — обеспечивает подключение к WebSocket и динамическое обновление таблицы. Краткий листинг кода показан на рисунке 4.

```
1 let ws = null;
2 let observerId = null;
3
4 const currencyNames = {
5   'USD': 'Доллар США',
6   'EUR': 'Евро',
7   'GBP': 'Фунт стерлингов',
8   'CNY': 'Китайский юань',
9   'JPY': 'Японская иена',
10  'RUB': 'Российский рубль'
11 };
12
13 function connectWebSocket() {
14   if (ws && ws.readyState === WebSocket.OPEN) {
15     console.log('Уже подключен');
16     return;
17   }
18
19   // Создаем WebSocket соединение
20   const protocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
21   const wsUrl = `${protocol}://${window.location.host}/ws`;
22
23   ws = new WebSocket(wsUrl);
24
25   ws.onopen = function() {
```

Рисунок 4. Листинг файла client.js

Файл index.html — определяет структуру пользовательского интерфейса, содержит таблицу для отображения курсов и элементы управления. Часть листинга кода показана на рисунке 5.

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Отслеживание курсов валют</title>
7   <style>
8     body {
9       font-family: Arial, sans-serif;
10      margin: 20px;
11      background-color: #f5f5f5;
12    }
13
14    .container {
15      max-width: 800px;
16      margin: 0 auto;
17      background: white;
18      padding: 20px;
19      border-radius: 10px;
20      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
21    }
22
23    h1 {
24      color: #333;
25      text-align: center;
26    }
```

Рисунок 5. Листинг файла index.html

Полный листинг всех вышеперечисленных кодов представлен в приложении А.

2.3. Контейнеризация и развёртывание

Файл Dockerfile — определяет процесс сборки образа. Код файла представлен ниже, на рисунке 6.

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY currency_observer/ ./currency_observer/
9
10 CMD ["python", "-m", "currency_observer.main"]
```

Рисунок 6. Листинг файла Dockerfile

Файл docker-compose.yml — описывает оркестрацию трёх сервисов. Листинг кода этого файла показан на рисунке 7.

```
services:
  currency-app:
    build: .
    container_name: currency-app
    restart: unless-stopped
    expose:
      - "8888"
    environment:
      - VIRTUAL_HOST=stepanna.bizml.ru,www.stepanna.bizml.ru
      - VIRTUAL_PORT=8888
      - LETSENCRYPT_HOST=stepanna.bizml.ru,www.stepanna.bizml.ru
      - LETSENCRYPT_EMAIL=${EMAIL:-annastep439@gmail.com}
    networks:
      - proxy-network

  # NGINX Reverse Proxy
  nginx-proxy:
    image: nginxproxy/nginx-proxy:latest
    container_name: nginx-proxy
    restart: unless-stopped
    ports:
      - "8880:80"
      - "8443:443"
    volumes:
      - /var/run/docker.sock:/tmp/docker.sock:ro
      - ./certs:/etc/nginx/certs
      - ./vhost.d:/etc/nginx/vhost.d
      - ./html:/usr/share/nginx/html
    networks:
```

Рисунок 7. Листинг файла docker-compose.yml

Полный листинг вышеперечисленных кодов будет представлен в приложении Б.

Развёртывание разработанного приложения производилось на VPS-сервере под управлением операционной системы Ubuntu 22.04. Процесс развёртывания включал несколько последовательных этапов. На первом этапе выполнялась подготовка сервера, а именно установка платформы Docker и инструмента оркестрации Docker Compose. После этого осуществлялось копирование проекта — перенос всех необходимых файлов с локальной машины разработчика на удалённый сервер. Следующим шагом стала настройка DNS: для домена stepanna.bizml.ru была создана A-запись, указывающая на публичный IP-адрес сервера (188.127.240.135). Затем производился запуск приложения с помощью команды `docker compose up -d`. На заключительном этапе сработал автоматический механизм получения SSL-сертификата: сервис acme-companion самостоятельно запросил и установил сертификат Let's Encrypt, после чего приложение стало доступно по защищённому протоколу HTTPS.

Для корректной работы WebSocket-соединений в конфигурацию Nginx были добавлены специальные заголовки, показанные на рисунке 8.

```
location /ws {  
    proxy_pass http://127.0.0.1:8888;  
    proxy_http_version 1.1;  
    proxy_set_header Upgrade $http_upgrade;  
    proxy_set_header Connection "upgrade";  
}
```

Рисунок 8. Листинг конфигурации Nginx

2.4. Результаты работы

На рисунке 9 представлен скриншот работающего приложения по адресу <https://stepanna.bizml.ru>.

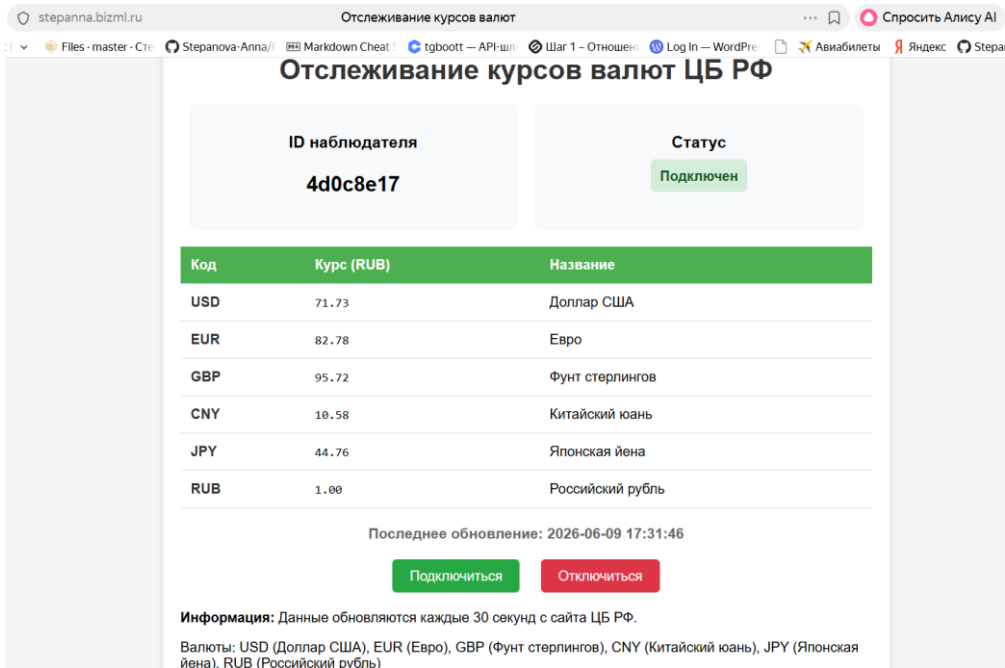


Рисунок 9. Главная страница приложения с отображаемыми курсами валют

На рисунке 9 представлен скриншот главной страницы работающего приложения по адресу <https://stepanna.bizml.ru>. Как показано на рисунке, в адресной строке браузера отображается значок защищённого соединения, то есть соединение защищено сертификатом, выданным удостоверяющим центром Let's Encrypt для домена stepanna.bizml.ru.

На рисунке 10 представлен вывод команды `docker ps`. Он демонстрирует, что все три сервиса (`currency-app`, `nginx-проху` и `nginx-letsencrypt`) находятся в рабочем состоянии.

NAMES	STATUS	PORTS
nginx-letsencrypt	Up 53 minutes	
nginx-proxy	Up 36 minutes	0.0.0.0:8880->80/tcp, [::]:8880->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
currency-app	Up 11 minutes	8888/tcp

Рисунок 10. Состояние запущенных Docker-контейнеров

2.5. Выводы по практической части

В ходе практической реализации были достигнуты следующие результаты. Прежде всего, разработано полноценное веб-приложение для отслеживания курсов валют, в котором для передачи данных в реальном времени используется протокол WebSocket, обеспечивающий оперативное обновление

информации без перезагрузки страницы. Далее была настроена контейнеризация приложения с помощью инструментов Docker и Docker Compose, что позволило обеспечить изоляцию каждого компонента системы, а также воспроизводимость окружения независимо от платформы развёртывания. Кроме того, организован безопасный доступ к приложению по протоколу HTTPS, причём процесс получения и обновления SSL-сертификатов от центра сертификации Let's Encrypt полностью автоматизирован. В итоге приложение успешно развёрнуто на VPS-сервере и доступно для использования по адресу <https://stepanna.bizml.ru>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы была достигнута поставленная цель: разработано и развёрнуто веб-приложение для отслеживания курсов валют в реальном времени с автоматическим получением и обновлением SSL-сертификатов.

В рамках теоретической части работы были изучены основные принципы контейнеризации на платформе Docker, рассмотрены понятия образа и контейнера, а также инструмент оркестрации Docker Compose. Особое внимание было уделено протоколу WebSocket, обеспечивающему полнодуплексную связь между клиентом и сервером, что критически важно для приложений реального времени. Также был проведён анализ способов автоматизации SSL/TLS с использованием бесплатного центра сертификации Let's Encrypt и протокола ACME. Обоснован выбор связки nginx-proxy и acme-companion как наиболее простого и надёжного решения для автоматического получения и обновления сертификатов.

В практической части работы было разработано клиент-серверное веб-приложение на языке Python с использованием фреймворка Tornado, обеспечивающее как обработку HTTP-запросов, так и поддержку WebSocket-соединений. Серверная часть приложения периодически (каждые 30 секунд) обращается к официальному API Центрального банка Российской Федерации для получения актуальных курсов валют, после чего рассылает обновления всем подключённым клиентам через WebSocket. Клиентская часть, реализованная на HTML5, CSS3 и JavaScript, отображает полученные данные в виде таблицы и динамически обновляет их без перезагрузки страницы.

Для обеспечения воспроизводимости окружения и изоляции компонентов была настроена контейнеризация приложения с помощью Docker и Docker Compose. Проект включает три сервиса: непосредственно само приложение currency-app, reverse-proxy nginx-proxy для маршрутизации

трафика и терминирования SSL, а также `acme-companion` для автоматического управления сертификатами. Развёртывание производилось на VPS-сервере под управлением Ubuntu 22.04. Для домена `stepanna.bizml.ru` была настроена А-запись, указывающая на IP-адрес сервера.

Особое внимание в работе уделено обеспечению безопасности. Благодаря использованию связки `nginx-proxy` и `acme-companion`, процесс получения и обновления SSL-сертификатов от Let's Encrypt полностью автоматизирован и не требует ручного вмешательства. Для корректной работы WebSocket через `reverse-proxy` в конфигурацию Nginx были добавлены специальные заголовки, обеспечивающие обновление протокола с HTTP на WebSocket.

В результате проделанной работы приложение успешно функционирует по адресу `https://stepanna.bizml.ru`, обеспечивая отображение актуальных курсов шести валют (USD, EUR, GBP, CNY, JPY, RUB) с автоматическим обновлением каждые 30 секунд. Все соединения защищены протоколом HTTPS, о чём свидетельствует действующий сертификат Let's Encrypt.

Таким образом, все поставленные в начале работы задачи были выполнены в полном объёме. Разработанное решение может использоваться в качестве шаблона для развёртывания других веб-приложений, требующих работы в реальном времени и автоматического HTTPS. Перспективами дальнейшего развития проекта могут стать добавление большего количества валют, реализация графиков изменения курсов, настройка системы уведомлений о значительных колебаниях, а также внедрение системы кэширования для снижения нагрузки на внешний API.

СПИСОК ЛИТЕРАТУРЫ

1. Введение в Let's Encrypt: бесплатные SSL-сертификаты. — Текст : электронный // arenda-server : [сайт]. — URL: <https://arenda-server.cloud/blog/vvedenie-v-let-s-encrypt-besplatnye-ssl-sertifikaty/> (дата обращения: 08.06.2026).
2. Дюгуров, Д. В. Контейнеризация. Ч. 1 : Введение в Docker : учеб.-метод. пособие / Д. В. Дюгуров. — Ижевск : Удмуртский ун-т, 2023. — 47 с. — Текст : электронный.
3. Контейнеризатор приложений docker — установка, настройка, основы управления приложениями в средах с поддержкой контейнеризации . — Текст : электронный // Лань : [сайт]. — URL: <https://e.lanbook.com/book/460064?category=931> (дата обращения: 08.06.2026).
4. Микросервисная архитектура как современный подход к разработке программного обеспечения. — Текст : электронный // cyberleninka.ru : [сайт]. — URL: <https://cyberleninka.ru/article/n/mikroservisnaya-arhitektura-kak-sovremennyy-podhod-k-razrabotke-programmnogo-obespecheniya> (дата обращения: 08.06.2026).
5. Настройка и администрирование сервисного программного обеспечения: учебное пособие. — Текст : электронный // Лань : [сайт]. — URL: <https://e.lanbook.com/book/382442?category=1537> (дата обращения: 08.06.2026).
6. Официальные курсы валют на заданную дату, устанавливаемые ежедневно. — Текст : электронный // Банк России : [сайт]. — URL: https://www.cbr.ru/currency_base/ (дата обращения: 08.06.2026).

7. acme-companion. — Текст : электронный // github.com : [сайт]. — URL: <https://github.com/nginx-proxy/acme-companion> (дата обращения: 08.06.2026).
8. Docker. — Текст : электронный // skillfactory : [сайт]. — URL: <https://blog.skillfactory.ru/glossary/docker/> (дата обращения: 08.06.2026).
9. Docker Compose. — Текст : электронный // dockerdocs : [сайт]. — URL: <https://docs.docker.com/compose/> (дата обращения: 08.06.2026).
10. Documentation. — Текст : электронный // Let's Encrypt : [сайт]. — URL: <https://letsencrypt.org/docs/> (дата обращения: 08.06.2026).
11. Nginx. Reverse Proxy. — Текст : электронный // sysadminium : [сайт]. — URL: <https://blog.sysadminium.ru/docs/web/nginx-reverse-proxy/> (дата обращения: 08.06.2026).
12. WebSocket.org. — Текст : электронный // websocket : [сайт]. — URL: <https://websocket.org/> (дата обращения: 08.06.2026).

ПРИЛОЖЕНИЕ А

Листинг кода файла currency_service.py

```
import asyncio
import aiohttp
from typing import Dict, Any, Optional
from datetime import datetime, timedelta
import json

class CurrencyService:
    """Сервис для получения курсов валют с API ЦБ РФ"""

    API_URL = "https://www.cbr-xml-daily.ru/daily_json.js"

    def __init__(self, update_interval: int = 300) -> None:
        self.update_interval = update_interval
        self.last_update: Optional[datetime] = None
        self.currencies: Dict[str, float] = {}

    async def fetch_currency_rates(self) -> Dict[str, float]:
        """Получить курсы валют с API"""
        try:
            async with aiohttp.ClientSession() as session:
                headers = {
                    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36'
                }
                async with session.get(self.API_URL, headers=headers,
timeout=10) as response:
                    if response.status == 200:
                        data = await response.json(content_type=None)
                        return self._parse_currency_data(data)
                    else:
                        print(f"Ошибка API: {response.status}")
                        return {}
        except Exception as e:
            print(f"Ошибка при получении данных: {e}")
            return {}

    def _parse_currency_data(self, data: Dict[str, Any]) -> Dict[str, float]:
        """Парсинг данных о валютах"""
        currencies = {}

        target_currencies = ['USD', 'EUR', 'GBP', 'CNY', 'JPY']

        if 'Valute' in data:
            for currency_code, currency_info in data['Valute'].items():
                if currency_code in target_currencies:
                    currencies[currency_code] = currency_info['Value']

        currencies['RUB'] = 1.0

        return currencies

    def should_update(self) -> bool:
        """Проверка, нужно ли обновлять данные"""
```

```

        if not self.last_update:
            return True

        time_since_update = datetime.now() - self.last_update
        return time_since_update.total_seconds() >= self.update_interval

    async def get_updated_rates(self) -> Dict[str, float]:
        """Получить обновленные курсы валют"""
        if self.should_update():
            new_rates = await self.fetch_currency_rates()
            if new_rates:
                self.currencies = new_rates
                self.last_update = datetime.now()
                print(f"Курсы обновлены: {datetime.now().strftime('%H:%M:%S')}")
                for currency, rate in self.currencies.items():
                    if currency != 'RUB':
                        print(f"    {currency}: {rate:.2f} RUB")
            else:
                print("Не удалось получить новые курсы валют")

        return self.currencies

    def set_update_interval(self, interval: int) -> None:
        """Установить интервал обновления (в секундах)"""
        self.update_interval = interval
        print(f"Интервал обновления установлен: {interval} секунд")

```

Листинг кода файла observer.py

```

from typing import List, Dict, Any, Protocol, Optional
import json
import uuid
from datetime import datetime

class Observer(Protocol):
    """Протокол для наблюдателей"""
    observer_id: str

    async def update(self, currency_data: Dict[str, Any]) -> None:
        """Метод для обновления данных у наблюдателя"""
        pass

class CurrencySubject:
    """Субъект, который отслеживает изменения курсов валют"""

    def __init__(self) -> None:
        self._observers: List[Observer] = []
        self._currency_data: Dict[str, float] = {}

    def attach(self, observer: Observer) -> None:
        """Добавить наблюдателя"""
        if observer not in self._observers:
            self._observers.append(observer)
            print(f"Наблюдатель {observer.observer_id} подключен")

    def detach(self, observer: Observer) -> None:
        """Удалить наблюдателя"""
        if observer in self._observers:

```



```

        self._observers.remove(observer)
        print(f"Наблюдатель {observer.observer_id} отключен")

    async def notify(self) -> None:
        """Уведомить всех наблюдателей об изменениях"""
        data = {
            'currencies': self._currency_data,
            'timestamp': self._get_current_timestamp()
        }

        for observer in self._observers:
            try:
                await observer.update(data)
            except Exception as e:
                print(f"Ошибка при уведомлении наблюдателя {observer.observer_id}: {e}")

    def set_currency_data(self, new_data: Dict[str, float]) -> None:
        """Установить новые данные о курсах валют"""
        self._currency_data = new_data

    def _get_current_timestamp(self) -> str:
        """Получить текущее время в строковом формате"""
        return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    @property
    def observer_count(self) -> int:
        """Количество активных наблюдателей"""
        return len(self._observers)

class WebSocketObserver:
    """Наблюдатель, реализующий WebSocket соединение"""

    def __init__(self, websocket) -> None:
        self.websocket = websocket
        self.observer_id = str(uuid.uuid4())[:8]

    async def update(self, currency_data: Dict[str, Any]) -> None:
        """Отправить обновление через WebSocket"""
        try:
            message = {
                'type': 'currency_update',
                'data': currency_data,
                'observer_id': self.observer_id
            }
            await self.websocket.write_message(json.dumps(message))
        except Exception as e:
            print(f"Ошибка отправки сообщения наблюдателю {self.observer_id}: {e}")

```

Листинг кода файла client.js

```

let ws = null;
let observerId = null;

const currencyNames = {
    'USD': 'Доллар США',
    'EUR': 'Евро',
    'GBP': 'Фунт стерлингов',
    'CNY': 'Китайский юань',

```

```

    'JPY': 'Японская иена',
    'RUB': 'Российский рубль'
};

function connectWebSocket() {
    if (ws && ws.readyState === WebSocket.OPEN) {
        console.log('Уже подключен');
        return;
    }

    // Создаем WebSocket соединение
    const protocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
    const wsUrl = `${protocol}://${window.location.host}/ws`;

    ws = new WebSocket(wsUrl);

    ws.onopen = function() {
        console.log('WebSocket соединение установлено');
        updateConnectionStatus(true);
        document.getElementById('connect-btn').disabled = true;
        document.getElementById('disconnect-btn').disabled = false;
    };

    ws.onclose = function() {
        console.log('WebSocket соединение закрыто');
        updateConnectionStatus(false);
        document.getElementById('connect-btn').disabled = false;
        document.getElementById('disconnect-btn').disabled = true;
        observerId = null;
    };

    ws.onerror = function(error) {
        console.error('WebSocket ошибка:', error);
        updateConnectionStatus(false);
    };

    ws.onmessage = function(event) {
        const data = JSON.parse(event.data);
        console.log('Получены данные:', data);

        if (data.type === 'connection_established') {
            observerId = data.observer_id;
            document.getElementById('observer-id').textContent = observerId;
            console.log(`ID наблюдателя: ${observerId}`);
        }
        else if (data.type === 'currency_update') {
            updateCurrencyTable(data.data);
        }
    };
}

function disconnectWebSocket() {
    if (ws) {
        ws.close();
        ws = null;
    }
}

function updateConnectionStatus(connected) {
    const statusElement = document.getElementById('connection-status');
    const statusText = document.getElementById('connection-status');

    if (connected) {

```

```

        statusText.textContent = 'Подключен';
        statusElement.className = 'status connected';
    } else {
        statusText.textContent = 'Отключен';
        statusElement.className = 'status disconnected';
    }
}

function updateCurrencyTable(data) {
    const tbody = document.getElementById('currency-data');
    const currencies = data.currencies || {};
    const timestamp = data.timestamp || '-';

    document.getElementById('last-update').textContent = timestamp;

    tbody.innerHTML = '';

    const sortedCurrencies = Object.entries(currencies).sort(([codeA],
[codeB]) => {
        const order = ['USD', 'EUR', 'GBP', 'CNY', 'JPY', 'RUB'];
        return order.indexOf(codeA) - order.indexOf(codeB);
    });

    sortedCurrencies.forEach(([code, rate]) => {
        const row = document.createElement('tr');

        const codeCell = document.createElement('td');
        codeCell.className = 'currency-code';
        codeCell.textContent = code;

        const rateCell = document.createElement('td');
        rateCell.className = 'currency-rate';
        rateCell.textContent = code === 'RUB' ? '1.00' : rate.toFixed(2);

        const nameCell = document.createElement('td');
        nameCell.textContent = currencyNames[code] || code;

        row.appendChild(codeCell);
        row.appendChild(rateCell);
        row.appendChild(nameCell);

        tbody.appendChild(row);
    });
}

// Автоподключение при загрузке страницы
window.addEventListener('DOMContentLoaded', function() {
    connectWebSocket();

    // Автоматическое переподключение при потере соединения
    setInterval(function() {
        if (ws && ws.readyState === WebSocket.CLOSED) {
            console.log('Попытка переподключения...');
            connectWebSocket();
        }
    }, 5000);
});

```

Листинг кода файла index.html

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Отслеживание курсов валют</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background-color: #f5f5f5;
    }

    .container {
      max-width: 800px;
      margin: 0 auto;
      background: white;
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }

    h1 {
      color: #333;
      text-align: center;
    }

    .status {
      padding: 10px;
      border-radius: 5px;
      margin: 10px 0;
      text-align: center;
      font-weight: bold;
    }

    .connected {
      background-color: #d4edda;
      color: #155724;
    }

    .disconnected {
      background-color: #f8d7da;
      color: #721c24;
    }

    .currency-table {
      width: 100%;
      border-collapse: collapse;
      margin: 20px 0;
    }

    .currency-table th,
    .currency-table td {
      padding: 12px;
      text-align: left;
      border-bottom: 1px solid #ddd;
    }

    .currency-table th {
      background-color: #4CAF50;
      color: white;
    }
  </style>
</head>
</html>

```

```

.currency-table tr:hover {
    background-color: #f5f5f5;
}

.currency-code {
    font-weight: bold;
    color: #333;
}

.currency-rate {
    font-family: monospace;
    font-size: 1.1em;
}

.last-update {
    text-align: center;
    color: #666;
    font-size: 0.9em;
    margin-top: 20px;
}

.observer-info {
    background-color: #e9ecef;
    padding: 10px;
    border-radius: 5px;
    margin: 10px 0;
}

.control-buttons {
    text-align: center;
    margin: 20px 0;
}

button {
    padding: 10px 20px;
    margin: 0 10px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-size: 16px;
}

#connect-btn {
    background-color: #28a745;
    color: white;
}

#disconnect-btn {
    background-color: #dc3545;
    color: white;
}

button:disabled {
    background-color: #6c757d;
    cursor: not-allowed;
}

.stats {
    display: flex;
    justify-content: space-around;
    margin: 20px 0;
}

```

```

        .stat-card {
            background: #f8f9fa;
            padding: 15px;
            border-radius: 8px;
            text-align: center;
            flex: 1;
            margin: 0 10px;
        }
    }
</style>
</head>
<body>
    <div class="container">
        <h1>Отслеживание курсов валют ЦБ РФ</h1>

        <div class="stats">
            <div class="stat-card">
                <h3>ID наблюдателя</h3>
                <p id="observer-id" style="font-size: 24px; font-weight:
bold;">Не подключен</p>
            </div>
            <div class="stat-card">
                <h3>Статус</h3>
                <span id="connection-status" class="status
disconnected">Отключен</span>
            </div>
        </div>

        <table class="currency-table">
            <thead>
                <tr>
                    <th>Код</th>
                    <th>Курс (RUB)</th>
                    <th>Название</th>
                </tr>
            </thead>
            <tbody id="currency-data">
                <tr>
                    <td colspan="3" style="text-align: center; padding:
40px;">Загрузка данных...</td>
                </tr>
            </tbody>
        </table>

        <div class="last-update">
            <h3>Последнее обновление: <span id="last-update">—</span></h3>
        </div>

        <div class="control-buttons">
            <button id="connect-btn"
onclick="connectWebSocket()">Подключиться</button>
            <button id="disconnect-btn" onclick="disconnectWebSocket()"
disabled>Отключиться</button>
        </div>

        <div class="info">
            <p><strong>Информация:</strong> Данные обновляются каждые 30
секунд с сайта ЦБ РФ.</p>
            <p>Валюты: USD (Доллар США), EUR (Евро), GBP (Фунт стерлингов),
CNY (Китайский юань), JPY (Японская йена), RUB (Российский рубль)</p>
        </div>
    </div>

    <script>

```

```

let ws = null;
let observerId = null;

const currencyNames = {
  'USD': 'Доллар США',
  'EUR': 'Евро',
  'GBP': 'Фунт стерлингов',
  'CNY': 'Китайский юань',
  'JPY': 'Японская йена',
  'RUB': 'Российский рубль'
};

function connectWebSocket() {
  if (ws && (ws.readyState === WebSocket.OPEN || ws.readyState ===
WebSocket.CONNECTING)) {
    console.log('Уже подключен или подключается');
    return;
  }

  const protocol = window.location.protocol === 'https:' ? 'wss:' :
'ws: ';
  const wsUrl = `${protocol}//${window.location.host}/ws`;

  console.log('Подключение к:', wsUrl);
  ws = new WebSocket(wsUrl);

  ws.onopen = function() {
    console.log('WebSocket соединение установлено');
    updateConnectionStatus(true);
    document.getElementById('connect-btn').disabled = true;
    document.getElementById('disconnect-btn').disabled = false;
  };

  ws.onclose = function(event) {
    console.log('WebSocket соединение закрыто, код:', event.code,
'причина:', event.reason);
    updateConnectionStatus(false);
    document.getElementById('connect-btn').disabled = false;
    document.getElementById('disconnect-btn').disabled = true;
    observerId = null;
  };

  ws.onerror = function(error) {
    console.error('WebSocket ошибка:', error);
    updateConnectionStatus(false);
  };

  ws.onmessage = function(event) {
    try {
      const data = JSON.parse(event.data);
      console.log('Получены данные:', data);

      if (data.type === 'connection_established') {
        observerId = data.observer_id;
        document.getElementById('observer-id').textContent =
observerId;

        console.log(`ID наблюдателя: ${observerId}`);
      } else if (data.type === 'currency_update') {
        updateCurrencyTable(data.data);
      }
    } catch (e) {
      console.error('Ошибка парсинга JSON:', e, 'Полученные

```

```

данные:', event.data);
    }
    };
}

function disconnectWebSocket() {
    if (ws) {
        ws.close();
        ws = null;
    }
}

function updateConnectionStatus(connected) {
    const statusElement = document.getElementById('connection-
status');
    const statusText = document.getElementById('connection-status');

    if (connected) {
        statusText.textContent = 'Подключен';
        statusElement.className = 'status connected';
    } else {
        statusText.textContent = 'Отключен';
        statusElement.className = 'status disconnected';
    }
}

function updateCurrencyTable(data) {
    const tbody = document.getElementById('currency-data');
    const currencies = data.currencies || {};
    const timestamp = data.timestamp || '-';

    document.getElementById('last-update').textContent = timestamp;

    tbody.innerHTML = '';

    const sortedCurrencies =
Object.entries(currencies).sort(([codeA], [codeB]) => {
    const order = ['USD', 'EUR', 'GBP', 'CNY', 'JPY', 'RUB'];
    return order.indexOf(codeA) - order.indexOf(codeB);
}));

    if (sortedCurrencies.length === 0) {
        const row = document.createElement('tr');
        const cell = document.createElement('td');
        cell.colSpan = 3;
        cell.style.textAlign = 'center';
        cell.style.padding = '40px';
        cell.textContent = 'Данные о курсах валют отсутствуют';
        row.appendChild(cell);
        tbody.appendChild(row);
        return;
    }

    sortedCurrencies.forEach(([code, rate]) => {
        const row = document.createElement('tr');

        const codeCell = document.createElement('td');
        codeCell.className = 'currency-code';

```



```

        codeCell.textContent = code;

        const rateCell = document.createElement('td');
        rateCell.className = 'currency-rate';
        rateCell.textContent = code === 'RUB' ? '1.00' :
rate.toFixed(2);

        const nameCell = document.createElement('td');
        nameCell.textContent = currencyNames[code] || code;

        row.appendChild(codeCell);
        row.appendChild(rateCell);
        row.appendChild(nameCell);

        tbody.appendChild(row);
    });
}

window.addEventListener('DOMContentLoaded', function() {
    console.log('Страница загружена, подключаемся к WebSocket...');
    connectWebSocket();

    // Автоматическое переподключение при потере соединения
    setInterval(function() {
        if (ws && ws.readyState === WebSocket.CLOSED) {
            console.log('Попытка переподключения...');
            connectWebSocket();
        }
    }, 5000);
});
</script>
</body>
</html>

```

ПРИЛОЖЕНИЕ Б

Листинг кода файла Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY currency_observer/ ./currency_observer/

CMD ["python", "-m", "currency_observer.main"]
```

Листинг кода файла docker-compose.yml

```
services:

  currency-app:
    build: .
    container_name: currency-app
    restart: unless-stopped
    expose:
      - "8888"
    environment:
      - VIRTUAL_HOST=stepanna.bizml.ru,www.stepanna.bizml.ru
      - VIRTUAL_PORT=8888
      - LETSENCRYPT_HOST=stepanna.bizml.ru,www.stepanna.bizml.ru
      - LETSENCRYPT_EMAIL=${EMAIL:-annastep439@gmail.com}
    networks:
      - proxy-network

  # NGINX Reverse Proxy
  nginx-proxy:
    image: nginxproxy/nginx-proxy:latest
    container_name: nginx-proxy
    restart: unless-stopped
    ports:
      - "8880:80"
      - "8443:443"
    volumes:
      - /var/run/docker.sock:/tmp/docker.sock:ro
      - ./certs:/etc/nginx/certs
      - ./vhost.d:/etc/nginx/vhost.d
      - ./html:/usr/share/nginx/html
    networks:
      - proxy-network

  # Let's Encrypt автоматический выпуск сертификатов
  letsencrypt:
    image: nginxproxy/acme-companion:latest
    container_name: nginx-letsencrypt
    restart: unless-stopped
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./certs:/etc/nginx/certs
      - ./vhost.d:/etc/nginx/vhost.d
      - ./html:/usr/share/nginx/html
```

```
- ./acme:/etc/acme.sh
environment:
- DEFAULT_EMAIL=${EMAIL:-annastep439@gmail.com}
- NGINX_PROXY_CONTAINER=nginx-proxy
depends_on:
- nginx-proxy
networks:
- proxy-network

networks:
  proxy-network:
    driver: bridge
```